LR-Agent / Agent 后端入门

第01章
读懂Agent所需的Python基础

不追求一次学完所有 Python。围绕后端的数据、函数、对象和执行方式，补齐阅读 Agent 源码真正需要的基础。

读者起点

只掌握最基本的 Python；LangChain 从零开始。每次先读一个小例子，运行并解释结果，再进入项目源码。

学完本章应该能够

- 说清列表、字典、JSON、函数返回值和对象字段的区别。
- 读懂类型标注、None、Pydantic、异常与常见装饰器写法。
- 建立 async/await 与 yield 的最小直觉，不误解成自动并行。
- 读懂 AgentContext 的一小段源码并准确复述含义。

范围：Python、LangChain 与 Agent 后端。外部调用方只作为接口参与者，不安排前端实现学习。
基线：2026-10-17，d5dd009；LangChain 1.4.2 / LangGraph 1.2.12。

本章目录与阅读方法

1.1 先让自己的代码运行起来 . . . 3

1.2 列表、字典与 JSON：任务怎样表示 . . . 4

1.3 函数、返回值与类型标注 . . . 5

1.4 None、条件与默认值 . . . 6

1.5 集合与完成 ID：先理解数据，再谈幂等 . . . 7

1.6 import 与路径：为什么 app 找不到 . . . 8

1.7 类、对象、self 与方法 . . . 9

1.8 Pydantic：把输入规则写出来 . . . 10

1.9 异常：让失败有清楚的边界 . . . 11

1.10 装饰器：在函数外加一层处理 . . . 12

1.11 async 与 await：先理解等待 . . . 13

1.12 yield：一次产出一个结果 . . . 14

1.13 回到项目：读懂 AgentContext . . . 15

1.14 回顾、练习与面试准备 . . . 16

每节按“具体问题 → 名词解释 → 小例子 → 输出解释 → 项目对应 → 练习”推进。若代码出现陌生语法，先返回第 1 章相应小节，不要求提前掌握整门 Python。

蓝色标题可通过目录或书签跳转。完整实验由读者自行新建文件，不依赖额外附件。源码节选用于阅读，不能直接当完整脚本运行。

1.1 先让自己的代码运行起来

一段 Python 代码需要解释器运行。虚拟环境是隔离依赖的目录；同一台电脑的两个 Python，可能安装了不同版本的 LangChain。所有实验统一使用项目正式环境。

```
Set-Location D:\Study\LR-Agent
$env:PYTHONPATH = Join-Path (Get-Location) 'LR-Agent\vendor\local-agent'
$env:LANGSMITH_TRACING = 'false'
$env:LANGCHAIN_TRACING_V2 = 'false'
$agentPy = 'D:\Study\LR-Agent\.venvs\langchain-main\Scripts\python.exe'
& $agentPy -c "import sys; print(sys.version.split()[0])"
```

在 PowerShell 中执行，最后一行正常输出 3.12.0。$agentPy 是本终端保存的解释器路径，后续用 & 调用它。终端关闭后要重新执行这组准备命令。

每个实验的文件怎样创建

例如实验 py_data：在 study 目录自行新建 my_py_data.py，复制该节全部代码，再运行给出的命令。PDF 不附带这些脚本，避免读者依赖看不到的辅助文件。

```
& $agentPy ".\study\my_py_data.py"
```

这条命令的工作目录是 D:\Study\LR-Agent。相对路径 .\study 指当前目录下的 study；文件不在此处就会出现“找不到文件”。不要把命令直接粘进 Python 的 >>> 提示符。

本章不需要远程服务

正式环境已准备好。实验不请求云模型，不启动数据库或外部调用方；出现导入错误时，先核对解释器与工作目录。

1.2 列表、字典与 JSON：任务怎样表示

List / 列表：按顺序保存多个值。**Dict / 字典**：通过键找到值。两张图片组成列表，一次任务的路径、标签等字段组成字典。JSON 则是便于传输的文本格式。

完整教学实验 py_data。先在 study 中自行新建 my_py_data.py，复制全部代码。

```python
import json

paths = ["images/a.jpg", "images/b.jpg"]
task = {"paths": paths, "label": "car"}
print("first:", paths[0])
print("count:", len(task["paths"]))
text = json.dumps(task)
restored = json.loads(text)
print("json type:", type(text).__name__)
print("restored label:", restored["label"])
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```powershell
& $agentPy ".\study\my_py_data.py"
```

本轮实际输出：

```
first: images/a.jpg
count: 2
json type: str
restored label: car
```

关键行：paths[0] 取第一个元素，编号从 0 开始；task["paths"] 按键取列表；len 计算元素数量。dumps 将对象转成字符串，loads 把 JSON 字符串解析回对象。

项目对应：请求中的 messages 是消息列表，工具 arguments 常是字典，外部工具结果可能是 JSON 字符串。字符串看起来像字典，也必须先解析后才能按字段读取。

练习：给 paths 增加一张图片，预测 count 如何变化。这里没有打开任何图片，数量来自输入列表。

1.3 函数、返回值与类型标注

Function / 函数：把输入交给一段可重复使用的处理逻辑，再通过 return 返回结果。参数名 paths 只在函数的本次调用中指向传入的数据。

完整教学实验 py_function。先在 study 中自行新建 my_py_function.py，复制全部代码。

```python
def count_images(paths: list[str]) -> int:
    count = len(paths)
    return count

result = count_images(["a.jpg", "b.jpg"])
print("result:", result)
print("type:", type(result).__name__)
print("annotation is not validation:", count_images("abc"))
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```powershell
& $agentPy ".\study\my_py_function.py"
```

本轮实际输出：

```
result: 2
type: int
annotation is not validation: 3
```

逐行看：def 定义函数；paths: list[str] 表示“希望收到字符串列表”；-> int 表示“希望返回整数”；return count 把结果交给调用者。type(result).__name__ 读取类型名称；print 只负责显示，不能代替 return。

最后一行为什么得到 3

Python 通常不会凭类型标注自动拒绝错误输入。字符串也有 len，因此 "abc" 得到长度 3。要约束 API 输入，需要后面介绍的显式校验。

面试练习：“类型标注与输入校验有什么区别？”参考思路：标注描述开发者意图，校验在运行时检查数据是否满足规则；二者不能互相替代。

1.4 None、条件与默认值

项目常出现 str | None。它表示这个值可以是字符串，也可以是 None。None 表示“没有值”，与空字符串 "" 并不相同；是否允许它由业务规则决定。

完整教学实验 py_none。先在 study 中自行新建 my_py_none.py，复制全部代码。

```python
def describe_mode(mode: str | None) -> str:
    if mode is None:
        return "missing"
    if mode == "":
        return "empty"
    return mode

for value in [None, "", "assist"]:
    print(repr(value), "->", describe_mode(value))
print("fallback:", "" or "default")
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```powershell
& $agentPy ".\study\my_py_none.py"
```

本轮实际输出：

```
None -> missing
'' -> empty
'assist' -> assist
fallback: default
```

关键行：is None 专门判断缺失；== "" 判断空内容；return 会结束本次函数。repr 保留引号，方便观察空字符串。

表达式 x or fallback 会在 x 为假值时返回 fallback。空字符串、0、False 等都属于假值，因此它可能把不同含义合并。读取配置时要知道自己是否真的希望这样处理。

项目对应：AgentContext.workspace_root 可以为 None。没有工作区和工作区路径为空，需要在具体入口结合规则解释，不能见到 or 就认为校验完整。

1.5 集合与完成 ID：先理解数据，再谈幂等

Set / 集合：不重复保存元素，适合判断“这个 ID 是否已经出现”。**frozenset**：创建后不能增删元素的集合。不要依赖集合的遍历顺序。

完整教学实验 py_set。先在 study 中自行新建 my_py_set.py，复制全部代码。

```
completed_ids = {"call-001", "call-001"}
print("unique:", len(completed_ids))
print("known:", "call-001" in completed_ids)
print("new:", "call-002" not in completed_ids)
pending = ["call-001", "call-002"]
remaining = [item for item in pending if item not in completed_ids]
print("remaining:", remaining)
snapshot = frozenset(completed_ids)
print("snapshot size:", len(snapshot))
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```
& $agentPy ".\study\my_py_set.py"
```

本轮实际输出：

```
unique: 1
known: True
new: True
remaining: ['call-002']
snapshot size: 1
```

remaining 中的写法叫列表推导式，相当于遍历 pending，只把不在 completed_ids 中的 item 加入新列表。先用普通 for/if 理解，再读压缩写法。

项目对应：AgentContext.completed_tool_ids 表示本请求携带的完成记录。集合只是数据容器，真正“避免重复执行”还需要业务代码检查 ID 并采取一致的动作。

> **不要提前扩大结论**
>
> 本例没有并发、数据库和外部副作用；不能因为集合能去重，就声称整个系统具有分布式幂等保障。

1.6 import 与路径：为什么 app 找不到

Module / 模块：通常对应一个 .py 文件。**Package / 包**：组织相关模块的目录。import 让当前文件使用另一个模块中的定义，不是把代码从互联网下载下来。

```
from app.schemas.agent import ProposalStateInput
```

按层拆开：app 是包，schemas 是子目录，agent 指 agent.py，ProposalStateInput 是从文件里取出的类型。完整文件是 LR-Agent/vendor/local-agent/app/schemas/agent.py。

遇到的错误	先检查什么	为什么
No module named app	1.1 中的 PYTHONPATH 与目录	解释器要知道 app 的父目录
No module named langchain_core	是否用了 langchain-main 的 Python	依赖安装在具体虚拟环境中
No such file	study 文件名和当前工作目录	相对路径从当前目录开始计算

阅读源码时，先找定义再跟调用。导入一整个大模块可能触发其顶层代码，所以项目有些依赖采用函数内部的惰性导入；这不是要求初学者一开始就掌握的技巧。

练习：从 import 路径定位 app/agent/langchain_runtime/runtime.py，先只找到 class AgentContext，不必通读文件。

1.7 类、对象、self 与方法

Class / 类：描述某种对象有哪些数据和行为。**Instance / 实例**：按这个描述创建的具体对象。first 和 second 都属于 Task，但各自的 paths 不同。

完整教学实验 py_class。先在 study 中自行新建 my_py_class.py，复制全部代码。

```python
class Task:
    def __init__(self, paths: list[str]):
        self.paths = paths

    def count(self) -> int:
        return len(self.paths)

first = Task(["a.jpg", "b.jpg"])
second = Task(["c.jpg"])
print("first:", first.count())
print("second:", second.count())
print("first paths:", first.paths)
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```
& $agentPy ".\study\my_py_class.py"
```

本轮实际输出：

```
first: 2
second: 1
first paths: ['a.jpg', 'b.jpg']
```

__init__ 在创建对象时初始化字段；self 指当前这个对象；self.paths 保存当前对象的路径。count 是方法，first.count() 会在 first 的数据上执行。

后端常把多项输入保存在对象字段中，避免到处传散落的变量。第 2 章里的 message.content，就是从某个消息对象读取 content 字段。

练习：为什么 first.count() 和 second.count() 不同？请指出保存路径的字段和计算结果的方法。类型定义与对象本身也不是一回事。

1.8 Pydantic：把输入规则写出来

Pydantic 是用于数据模型与校验的库。class AnnotationTask(BaseModel) 表示继承 BaseModel 的能力，让字段声明参与初始化与校验。Field 用来附加默认值和边界规则。

完整教学实验 py_schema。先在 study 中自行新建 my_py_schema.py，复制全部代码。

```python
from pydantic import BaseModel, Field, ValidationError

class AnnotationTask(BaseModel):
    paths: list[str] = Field(min_length=1)
    confidence: float = Field(default=0.3, ge=0, le=1)

task = AnnotationTask(paths=["a.jpg", "b.jpg"])
print("count:", len(task.paths))
print("confidence:", task.confidence)
try:
    AnnotationTask(paths=["a.jpg"], confidence=1.2)
except ValidationError as exc:
    error = exc.errors()[0]
    print("invalid field:", error["loc"][0])
    print("error type:", error["type"])
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```powershell
& $agentPy ".\study\my_py_schema.py"
```

本轮实际输出：

```
count: 2
confidence: 0.3
invalid field: confidence
error type: less_than_equal
```

min_length=1 要求列表至少一项；ge=0 与 le=1 分别表示大于等于 0、小于等于 1。默认 confidence 为 0.3。错误信息准确指出 confidence 字段超出上界。

项目对应：app/schemas/agent.py 中的请求对象也使用 BaseModel 和 Field。本例 AnnotationTask 是教学模型，不是声称项目正式 Schema 长这样。

本节先照着运行 except；下一节解释异常处理。校验规则能否发现错误路径、是否允许写入文件，仍需要额外的业务检查。

1.9 异常：让失败有清楚的边界

无法把 "two" 转为整数时，int 会抛出 ValueError。try/except 让调用者区分“正常结果”和“失败原因”；它不等于忽略错误继续假装成功。

完整教学实验 py_error。先在 study 中自行新建 my_py_error.py，复制全部代码。

```python
def parse_count(text: str) -> int:
    try:
        return int(text)
    except ValueError:
        raise ValueError("count must be an integer") from None

for text in ["2", "two"]:
    try:
        print("count:", parse_count(text))
    except ValueError as exc:
        print("error:", str(exc))
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```powershell
& $agentPy ".\study\my_py_error.py"
```

本轮实际输出：

```
count: 2
error: count must be an integer
```

内层把底层错误转成更明确的说明，外层决定如何展示。raise 重新抛出；as exc 把异常对象绑定到变量。from None 只简化异常链显示，不改变失败事实。

项目对应：工具失败可能转成给模型看的结果、输出错误事件，或直接终止请求。具体策略在后续 Middleware 章节解释，不能把所有异常都吞掉后返回空值。

面试练习：为什么错误需要分层？参考：底层提供原因，业务层决定是否重试、降级或结束；不同失败不应都被描述为模型出错。

1.10 装饰器：在函数外加一层处理

装饰器不是神秘语法。@logged 放在函数定义上方，近似于定义后执行 count_images = logged(count_images)。下面的 logged 接收函数，再返回一个包着原函数的新函数。

完整教学实验 py_decorator。先在 study 中自行新建 my_py_decorator.py，复制全部代码。

```python
def logged(fn):
    def wrapper(paths):
        print("start")
        result = fn(paths)
        print("end")
        return result
    return wrapper

@logged
def count_images(paths):
    return len(paths)

print("count:", count_images(["a.jpg", "b.jpg"]))
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```powershell
& $agentPy ".\study\my_py_decorator.py"
```

本轮实际输出：

```
start
end
count: 2
```

定义外层函数时不会立即运行 wrapper。调用 count_images 时，才先打印 start，再调用保存的 fn，最后打印 end 并把原结果返回。变量 fn 指向传入的函数对象。

项目对应：后续 @tool 用装饰器把函数组织成工具对象，提供名称、描述与参数信息。这个 logged 只是解释包装思想，不能代替 LangChain 的工具实现。

这是完整但简化的教学包装器，只接收一个参数；通用装饰器中的 *args、**kwargs 和元信息保留，会在实际读到时再补。

1.11 async 与 await：先理解等待

async def 定义可异步执行的函数。调用它会得到协程对象，通常需要 await 等待结果。asyncio.run 启动一个事件循环，执行最外层的异步入口。

完整教学实验 py_async。先在 study 中自行新建 my_py_async.py，复制全部代码。

```python
import asyncio

async def read_result() -> str:
    print("read started")
    await asyncio.sleep(0)
    return "ready"

async def main():
    result = await read_result()
    print("result:", result)

asyncio.run(main())
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```
& $agentPy ".\study\my_py_async.py"
```

本轮实际输出：

```
read started
result: ready
```

main 等待 read_result，read_result 在 asyncio.sleep(0) 处让出控制机会，然后返回 ready。这里没有并行任务，因此输出顺序是固定的。

异步不等于自动提速

异步常用于等待网络和 I/O。只写 async 不会把 CPU 推理自动放到其他进程，也不能把阻塞调用自动变成非阻塞调用。

项目对应：ainvoke 通常等待一次异步结果，astream 则逐步接收异步流。第 5 章再结合取消与请求生命周期展开。

1.12 yield：一次产出一个结果

普通 return 结束函数；yield 暂停在当前位置并产出一个值，下次继续迭代时从暂停处接着走。带 yield 的函数返回生成器，适合逐步产生事件。

完整教学实验 py_yield。先在 study 中自行新建 my_py_yield.py，复制全部代码。

```python
def build_events():
    yield "preparing"
    yield "tool_pending"
    yield "done"

events = build_events()
print("first:", next(events))
for event in events:
    print("next:", event)
```

运行命令（沿用第 1 章 1.1 的 PowerShell 环境）：

```powershell
& $agentPy ".\study\my_py_yield.py"
```

本轮实际输出：

```
first: preparing
next: tool_pending
next: done
```

next 取走 preparing，之后的 for 继续得到剩下两项。它不会重新从第一项开始，也没有一次把所有事件都返回成列表。

项目对应：SSE 后端使用生成式输出组织进展。这里三个字符串只是教学数据，没有发送 HTTP，也不能认为字符串名称出现就已经执行了工具。

后续 async for 处理异步流，async def 内使用 yield 可构成异步生成器。现在先掌握“逐个产出”的含义，不必背完整异步协议。

1.13 回到项目：读懂 AgentContext

下面是 runtime.py::AgentContext 的源码节选，省略文档、导入和其他字段。它保存单次调用会使用的上下文信息，不是完整运行脚本。

```python
class AgentContext(BaseModel):
    mode: str = Field(default="assist", description="交互模式")
    client_tool_names: frozenset[str] = Field(
        default_factory=frozenset, description="由客户端执行的工具名"
    )
    workspace_root: str | None = Field(default=None, description="工作区根目录")

    def is_client_tool(self, name: str) -> bool:
        """该工具是否由客户端执行（后端不得执行）。"""
        return name in self.client_tool_names
```

现在可以逐项解释：BaseModel 提供数据模型能力；mode 默认是 assist；frozenset[str] 保存不重复的工具名称；default_factory=frozenset 每次创建默认集合；workspace_root 可以缺失；方法用 in 做归属判断。

语义静态与语言限制不同

AgentContext 被设计为本次调用的静态上下文，但模型没有 frozen=True。某个字段是 frozenset，也不代表整个对象的所有字段都不可修改。

完整路径：LR-Agent/vendor/local-agent/app/agent/langchain_runtime/runtime.py。此时只需读懂这个类的部分字段；Context 怎样传到工具，会在第 3 章解释。

1.14 回顾、练习与面试准备

练习 1：把实验中的 paths 改为三项，confidence 改为 1.2，分别预测行为。
参考：列表计数变为 3；Pydantic 在创建对象时拒绝超出范围的 confidence。

练习 2：口头解释 list[str]、str | None、self.paths、await、yield。
参考：期望的字符串列表；字符串或缺失；当前对象字段；等待异步结果；逐个产出值。

练习 3：阅读 AgentContext 后，是否能证明系统持久化了会话？
参考：不能。它描述单次调用中的信息；数据来源、保存位置和跨请求恢复要另外核对。

30 秒面试解释：类型标注和校验

“类型标注说明函数和字段期望的数据类型；运行时是否拒绝错误输入，取决于显式校验。本项目请求使用 Pydantic 校验字段，再由业务代码约束工具行为。我通过合法和越界输入分别观察了结果。”这是完成相应实验后的参考思路。

本章证据与通过标准

10 个完整实验已在正式环境运行。本章涉及教学数据与语言机制，没有实现新的业务工具。你应保存一次自己修改输入后的输出，并能定位请求 Schema 和 AgentContext。

进入下一章前

能看懂一个函数的输入与 return，能区分对象字段和字典键，能解释类的实例和异常。装饰器、异步仍不熟时可边学边回看，不需要一次背熟全部细节。

源码依据：app/schemas/agent.py、app/agent/langchain_runtime/runtime.py；路径均相对 LR-Agent/vendor/local-agent。下一章从模型消息开始，不直接进入大型 Agent 实现。